

SIH 2026

SYSTEM DESIGN SPECIFICATION

PostgreSQL Schema • Fabric Chaincode • Smart Contracts • QR/Scan/Event Flow

Base Model Engineering Design v1.0

This document freezes the database model, Fabric assets, smart-contract structure and the end-to-end scan/event behavior for the Base Model. It is the technical continuation of the PRD and TRD.

Critical project rule: every supply-chain stakeholder must be able to scan a product, see the permitted previous history, and have the scan/verified action auditable. A scan is recorded distinctly from a state-changing business event so repeated scans cannot falsely create transfers.

1. Design Baseline

The TRD freezes the core invariant as: **one stable product identity + one reliable lineage model + controlled lifecycle events**. PostgreSQL is the operational/query model; Redis is a cache; IPFS carries evidence; Fabric is the permissioned trust layer; chaincode enforces critical shared rules; and the Risk Propagator performs graph analysis outside chaincode.

■filecite■turn11file1■L1-L1■

The implementation must therefore avoid putting the complete application into Fabric. Fabric should hold the critical shared facts and auditable transaction/scan records required by the consortium. PostgreSQL should hold the query-optimized representation used by the UI and Risk Propagator.

2. Recommended Smart-Contract Count

For the Base Model, I recommend one Fabric chaincode package containing three logical smart contracts. This gives clean separation without creating a large number of independently deployed chaincodes.

Logical contract	Main responsibility	Why separate
TraceabilityContract	Product, batch, unit, lifecycle, receipt, transfer, transformation and lineage-related trusted state	Core supply-chain state machine
IncidentContract	Investigation, block, recall action, incident status and accountability/escalation commitments	Safety/action lifecycle
AuditContract	Stakeholder/consumer scan events, verification events and audit-only interaction records	Records scans without changing product state

These are logical contracts inside the same chaincode package. They can share utility functions and common ledger keys. This is preferable for the SIH Base Model to deploying three separate chaincode packages, because the operations share identities, state and transaction IDs.

The earlier TRD already defines the core chaincode transaction families — registerBatch, validateBatch, receiveBatch, transferBatch, createTransformation, createUnit, blockBatch/blockUnit, startInvestigation, createRecallAction and closeIncident — and explicitly keeps graph traversal, Redis, IPFS and analytics outside chaincode. ■filecite■turn11file16■L1-L1■

3. What Each Contract Must Never Do

Contract	Do NOT put inside
TraceabilityContract	Redis calls, IPFS uploads, Risk Propagator, complex graph queries, dashboard calculations
IncidentContract	Automatic legal fault determination, ML prediction, large evidence storage
AuditContract	Changing product ownership/state merely because a QR was scanned

4. PostgreSQL Schema — Core Relational Model

The following schema is the recommended Base Model. It is deliberately normalized around organizations, identities, products, batches, units, events, lineage, scans, incidents, evidence, accountability and risk scope. It supports the exact Base Model Definition of Done: source-to-consumer traceability, parent-child lineage, valid lifecycle transitions, feedback, Risk Propagation, recall, evidence, audit and queryability. ■filecite■turn11file2■L1-L1■

Table	Purpose	Key columns
organizations	Known supply-chain organizations	id PK, name, type, fabric_msp_id, status
users	Human/system actors	id PK, organization_id FK, role_id FK, auth_subject, status
roles	Role definitions	id PK, name
permissions	Fine-grained permissions	id PK, code
role_permissions	Role-permission mapping	role_id FK, permission_id FK
products	Product definitions	id PK, name, category, product_type, description
batches	Traceability lots/batches	id PK, product_id FK, code, quantity, unit, state, owner_org_id FK
units	Individually identifiable packages	id PK, batch_id FK, serial, state, qr_credential_id FK
lineage_edges	Parent-child relationships	id PK, parent_batch_id FK, child_batch_id FK, relation_type, quantity
custody_events	Movement/hand-over context	id PK, batch/unit, from_org, to_org, event_id FK
events	Operational representation of trusted business events	id PK, event_type, target_type, target_id, actor_org_id, actor_user_id, tx_id, timestamp
scan_events	Every auditable scan interaction	id PK, public_ref, entity_type, entity_id, scanner_type, actor_org_id, actor_user_id, session_ref, tx_id, timestamp
qr_credentials	Outer/inner credential references	id PK, unit_id FK, credential_type, public_ref, status, binding_ref
incidents	Consumer/stakeholder reports	id PK, unit_id/batch_id, category, severity, status, verification_status
feedback_evidence	Links feedback to evidence	incident_id FK, evidence_id FK
evidence	IPFS references	id PK, cid, content_hash, mime_type, size, created_by, created_at
accountability_records	Stakeholder accountability signals	id PK, incident_id, stakeholder_org_id, level, signal_value, reason, status
escalations	Escalation history	id PK, incident_id, from_org, to_org, trigger_rule, created_at
risk_scopes	Computed impact scope	id PK, incident_id, status, generated_at, generated_by
risk_scope_nodes	Nodes in an affected scope	scope_id FK, entity_type, entity_id, direction, impact_class
recall_actions	Authorized block/recall actions	id PK, incident_id, scope_id, action_type, status, authorized_by
audit_log	Application audit trail	id PK, actor, action, target, result, request_id, timestamp

5. Important PostgreSQL Relationships

1. organizations 1→N users; users N→1 roles
2. products 1→N batches
3. batches 1→N units
4. batches N↔N batches through lineage_edges (implemented as parent/child edges)
5. events can target a product, batch, unit or incident
6. scan_events reference the scanned public reference and resolved entity; stakeholder scans may carry organization/user identity
7. qr_credentials bind a unit to its outer/inner credential references
8. incidents can reference either a unit, a batch, or both
9. evidence is linked to incidents/events without storing large files in PostgreSQL
10. risk_scopes contain many risk_scope_nodes
11. recall_actions point to a reviewed risk scope

6. Batch vs Unit Rule

Use batch identity for most supply-chain operations. Create unit identities where individual consumer verification or precise recall requires them. The lineage graph remains batch-centric, while unit records connect final packages to their parent batch.

7. Fabric Ledger State Model

Fabric should contain compact, deterministic state objects representing the trusted shared view. PostgreSQL can contain richer operational metadata. The exact on-chain fields should be intentionally smaller than the relational schema.

Fabric key/type	Minimum trusted state
ProductAsset	productId, productType, status, createdByOrg, createdAt
BatchAsset	batchId, productId, quantity, state, ownerOrg, parentRefs, createdAt, updatedAt
UnitAsset	unitId, batchId, state, credentialRef, createdAt, updatedAt
IncidentAsset	incidentId, targetRef, severity, status, createdByOrg, createdAt, verificationState
RecallAsset	recallId, incidentId, scopeRef, status, authorizedBy, createdAt
ScanAuditAsset/Event	scanId, publicRef/entityRef, actorOrg/role or pseudonymous consumer ref, scanType, timestamp, result, txId

Do not put consumer personal data directly into the ledger. For consumer scans, use a pseudonymous session/reference if the product needs to count the scan. The consumer-facing identity should remain separate from private account information.

8. Smart Contract: TraceabilityContract

Function	Input	Validation / result
registerProduct	product metadata	Authorized org; unique product ID
registerBatch	product + batch + quantity	Authorized source/producer; unique batch; valid product type
validateBatch	batchId + validation result	Authorized validator; state must permit validation
receiveBatch	batchId + receiverOrg	Receiver authorized; batch transferable; append receipt event
transferBatch	batchId + toOrg	Sender authorized; valid state; update custody/owner
createTransformation	parentBatchIds + childBatch + quantities	Authorized processor; parent states valid; lineage/quantity rules
createUnit	batchId + unitId + credentialRef	Authorized packager/manufacturer; unit unique
updateAvailability	batch/unit	Authorized role; valid state
blockBatch / blockUnit	target + reason	Authorized role; state changes to BLOCKED

The TRD requires controlled lifecycle transitions and says chaincode must reject invalid transitions. A QR scan alone must not change state. ■filecite■turn11file8■L1-L1■

9. Smart Contract: IncidentContract

Function	Purpose
startInvestigation	Create/activate an investigation for a batch/unit incident
updateIncidentStatus	Move investigation through controlled states
recordEscalation	Record that accountability moved to an upstream layer
createRecallAction	Create authorized block/recall action against a reviewed scope
closeIncident	Close an incident after authorized resolution

Pattern detection and Risk Propagation are performed by the backend. Once a decision requires trusted shared recording — investigation, block, escalation or recall action — the backend submits the appropriate Fabric transaction.

10. Smart Contract: AuditContract

Function	Purpose
recordScan	Record an auditable scan interaction without changing product state
recordVerification	Record a successful/failed credential or traceability verification result
recordAuditEvent	Record selected critical non-state-changing interactions

This contract is the direct answer to the team requirement that every participant in the chain can scan and that the scan is auditable in blockchain. The scan record is intentionally separate from a transfer/receive event.

11. The Critical Scan Requirement — DO NOT MISS THIS

Yes — this requirement is part of the design and should be explicitly given to the implementation agent. Every relevant participant — producer, manufacturer, transporter/deliverer, retailer and consumer — should be able to scan the product/credential and receive the permitted previous history. At the same time, the system should create an auditable scan record so the platform knows that the product was scanned, by whom/which organization where appropriate, when, and what verification result occurred.

However, there is a crucial technical distinction: scan $\neq$ business event. If a manufacturer scans a product 10 times, the system must not create 10 transfers. The scan is recorded by AuditContract; the actual receive/transfer/transform operation is a separate authorized business transaction.

12. Scan Flow for Producer / Manufacturer / Transporter / Retailer

1. Stakeholder opens the scanner and scans the QR/credential
2. Backend resolves the public reference to batch/unit
3. Backend authenticates the stakeholder and identifies organization + role
4. System queries PostgreSQL/Redis for the current operational view and permitted previous history
5. System optionally queries Fabric for trusted current state / transaction proof
6. Backend submits recordScan to AuditContract
7. AuditContract records scanId, target reference, actor organization/role, scan type, timestamp and result
8. Fabric commits the scan audit event
9. Fabric event is consumed by backend and scan_events is updated in PostgreSQL
10. Redis cache is invalidated/refreshed if the scan changes a displayed summary
11. The stakeholder sees previous history and the current allowed action: receive, transfer, transform, verify, etc.

Then, if the stakeholder actually performs the business operation, a second transaction is submitted — for example receiveBatch or transferBatch. This creates the meaningful supply-chain event and changes state where appropriate.

13. Scan Flow for Consumer

1. Consumer scans outer QR
2. Backend resolves public reference to unit/batch
3. System returns consumer-permitted traceability history
4. AuditContract records a lightweight consumer scan event using a pseudonymous session/reference
5. Consumer may scan/enter the inner credential
6. Backend verifies the credential binding
7. AuditContract records verification result without exposing sensitive credential material
8. Consumer may submit feedback against the resolved unit/batch
9. Feedback is stored in PostgreSQL; evidence goes to IPFS if supplied
10. If the pattern becomes significant, the Risk Propagator and IncidentContract workflow can be triggered

Consumer scan data should not include unnecessary personal information on-chain. The system can record a pseudonymous scan/session reference, unit/batch reference, result and timestamp. This gives auditability without turning the ledger into a consumer surveillance database.

14. Scan Event vs Business Event — Exact Data Model

Dimension	Scan Event	Business Event
Meaning	User/system interacted with QR/credential	Authorized supply-chain operation occurred
Example	Manufacturer scanned B001	Manufacturer received B001
Changes product state?	No	Possibly yes
Fabric?	Yes, as AuditContract event in Base Model	Yes, through Traceability/IncidentContract
PostgreSQL?	scan_events + audit_log	events + affected entity tables
Can repeat freely?	Yes, subject to rate limits	Only if state/rules permit
Creates custody transfer?	No	Yes, if transfer operation
Creates lineage?	No	Yes, if transformation
Creates recall?	No	Yes, if authorized recall action

This separation is extremely important for Anti-Gravity/implementation. The system must not treat a raw QR scan as proof that custody changed. The scan proves an interaction; the business transaction proves an authorized action.

15. Previous History Returned on Every Stakeholder Scan

When a manufacturer/transporter/retailer scans, the UI should show the **previous permitted history** before allowing the next operation. A practical response should include: product identity, batch/unit, current state, previous custody events, previous transformations, source/parent summary, relevant validation results, warnings/block status, and the identity of the next expected operation.

Example manufacturer scan:

Wheat W001 → Flour F001 → received by Manufacturer M1
 Current state: RECEIVED
 Previous owner: Producer P1
 Validation: PASSED
 Previous event: Producer → Manufacturer
 Next allowed operation: PROCESS / TRANSFORM

Example transporter scan:

Biscuit Batch B001
 Current state: AVAILABLE / TRANSFERABLE
 Previous owner: Manufacturer M1
 Last event: Manufacturing completed
 Next allowed operation: TRANSFER TO RETAILER

This makes the scan a verification checkpoint rather than merely a QR lookup.

16. What Exactly Gets Stored on Fabric for a Scan

Field	Example	Notes
scanId	SCAN-8F21...	Unique transaction/event ID
targetRef	B001-U0723	Product/batch/unit reference
actorOrg	Retailer-R17	For known stakeholders
actorRole	RETAILER	Role at time of scan
scanType	TRACEABILITY / VERIFY / OPERATE	Purpose of scan
timestamp	server timestamp	Use trusted server/Fabric time context
result	FOUND / VERIFIED / BLOCKED / FAILED	Outcome
sessionRef	anonymous-session-91	Consumer pseudonymous reference
fabricTxId	...	Transaction proof
metadataHash	...	Optional integrity reference for extended metadata

Do not store raw GPS, consumer name, phone number, device fingerprint or other unnecessary personal information in Fabric. If future business-intelligence/location features require richer data, that belongs in a later module.

17. QR / Credential Tables and API Relationship

1. Outer QR → public_ref → qr_credentials → unit_id → batch_id → product_id
2. Unit/batch → events → previous history
3. Unit/batch → lineage_edges → upstream/downstream graph
4. Inner credential → credential verification service → unit binding status
5. Scan → AuditContract → scan_events
6. Business operation → TraceabilityContract → events + state
7. Incident → IncidentContract + PostgreSQL → Risk Propagator → risk_scope

18. Example: Complete Manufacturer Transaction

1. Manufacturer scans incoming Flour F001 QR
2. AuditContract records scan
3. System fetches Producer P1 → Wheat W001 → validation history
4. UI shows: F001 is derived from W001; current state = IN_TRANSIT; expected receiver = M1
5. Manufacturer selects RECEIVE
6. Backend validates application request
7. TraceabilityContract.receiveBatch(F001, M1) executes
8. Fabric commits the receive event
9. PostgreSQL stores the event + current state + fabricTxId
10. Redis refreshes F001 history
11. Manufacturer now sees F001 as RECEIVED under M1
12. If M1 later processes it, createTransformation creates the child batch and parent-child lineage

19. Example: Consumer → Feedback → Risk

1. Consumer scans B001-U0723 outer QR
2. AuditContract records scan
3. Consumer sees permitted history: W001 → F001 → B001
4. Consumer verifies inner credential
5. AuditContract records verification result
6. Consumer reports suspected contamination
7. PostgreSQL creates incident linked to U0723/B001
8. Optional evidence is stored in IPFS and linked to the incident
9. Pattern engine compares similar B001 complaints
10. If configured conditions are met, IncidentContract records escalation/investigation
11. Risk Propagator traverses B001 → F001 → W001 upstream and sibling/descendant scope downstream
12. Authorized stakeholder reviews the scope and can create block/recall action
13. Recall action is committed through Fabric and reflected in PostgreSQL

20. Chaincode Events and PostgreSQL Synchronization

Every successful Fabric transaction should emit a chaincode event containing enough information for the backend listener to update the operational model. PostgreSQL records the Fabric transaction ID so the read model can be reconciled against the trusted ledger.

1. Fabric transaction submitted
2. Endorsed + ordered + committed
3. Chaincode event emitted
4. Backend listener consumes event
5. PostgreSQL transaction updates affected rows
6. Redis invalidates affected keys
7. Audit/observability records synchronization result
8. If listener fails, event remains recoverable from Fabric and reconciliation retries

This follows the TRD's explicit consistency model: Fabric commit → event → PostgreSQL read model → Redis invalidation/refresh.

■filecite■turn11file16■L1-L1■

21. Suggested Repository / Service Structure

Component	Suggested modules
backend	auth, organizations, products, batches, units, events, qr, feedback, incidents, accountability, risk, recall, evidence, dashboard, fabric-gateway, fabric-events
chaincode	TraceabilityContract, IncidentContract, AuditContract, shared models, validators, access-control helpers
database	migrations, schema, indexes, seed data
risk-engine	lineage traversal, scope calculation, pattern evaluation
cache	Redis keys, invalidation, serialization
ipfs	upload, pinning, CID resolution, metadata
frontend	consumer scan, stakeholder scan, batch detail, lineage view, feedback, incident, recall, dashboards

22. Critical Indexes

At minimum, index **batches.code**, **units.serial**, **units.batch_id**, **lineage_edges.parent_batch_id**, **lineage_edges.child_batch_id**, **events.target_id + event_type**, **scan_events.public_ref**, **scan_events.entity_id + timestamp**, **incidents.batch_id/unit_id + category + created_at**, and **risk_scope_nodes.scope_id**. These are the main paths used by QR lookup, history, lineage and Risk Propagation.

23. Idempotency Rules

Business transactions must be idempotent where duplicate requests are possible. Every mutation should carry a client/request ID or deterministic operation reference. A repeated network retry must not create duplicate transfers, transformations, recall actions or units.

Scan events may legitimately repeat, but each scan gets a unique scanId/transaction ID and should be rate-limited. The implementation should distinguish duplicate business request from repeated scan interaction.

24. What Anti-Gravity Should Implement First

1. PostgreSQL migrations + schema + indexes
2. Organization/user/role seed data
3. Fabric network + MSP identities + Gateway
4. One chaincode package with three logical contracts
5. TraceabilityContract lifecycle and supply-chain functions
6. AuditContract recordScan + recordVerification
7. IncidentContract investigation/block/recall functions
8. Fabric event listener → PostgreSQL synchronization
9. QR resolution + stakeholder history view
10. Stakeholder scan → history → audit record → optional business operation
11. Consumer scan → history → verification → feedback
12. Lineage query + Risk Propagator
13. Basic recall/block dashboard

25. Non-Negotiable Acceptance Tests

Test	Expected result
Producer scans registered batch	Previous history shown; scan audit recorded
Manufacturer scans batch	Previous producer history shown; scan audit recorded; receive is a separate operation
Transporter scans batch	Previous history shown; scan audit recorded; transfer only after authorized action
Retailer scans batch/unit	Previous history shown; scan audit recorded
Consumer scans outer QR	Permitted traceability history shown; scan audit recorded without unnecessary PII
Consumer verifies inner credential	Authenticity result is separate from traceability result
Same stakeholder scans 5 times	5 scan interactions may be audited; product is not transferred 5 times

Test	Expected result
Invalid receive	Chaincode rejects transaction
Blocked batch scanned	History shows blocked status; prohibited business action rejected
Transformation	Child batch created and parent lineage preserved
Incident	Feedback linked to unit/batch
Repeated complaints	Pattern evaluation and configurable escalation work
Risk propagation	Upstream and downstream scope generated
Recall	Authorized targeted scope can be created and recorded
Fabric event listener failure	System can reconcile without inventing history

26. Final Decision

*For the Base Model, implement **one Fabric chaincode package with three logical contracts:***

***TraceabilityContract, IncidentContract and AuditContract.** Keep PostgreSQL as the operational/lineage model, Redis as cache, IPFS as evidence, and the Risk Propagator outside chaincode.*

Most importantly, the implementation must preserve the project story at every stakeholder checkpoint: scan → fetch previous permitted history → record the scan/audit event → if the stakeholder performs an authorized operation, record the separate business event and update state. This is the exact distinction that prevents the blockchain history from becoming incorrect while still giving us the auditable “every stakeholder scanned it” trail you want.

This document should be used as the direct technical baseline for implementation. The next level of detail, if required, is code-level: exact SQL DDL, Fabric asset JSON/state schemas, chaincode function signatures, API request/response schemas, and sequence diagrams for each transaction.